

Vision Transformer 이론 학습서

RNN부터 Vision Transformer까지의 완전한 이해

개요

딥러닝의 발전 과정에서 순환 신경망(RNN)에서 시작된 시퀀스 처리 기술이 어텐션 메커니즘을 거쳐 트랜스포머로 진화하였고, 최종적으로 이미지 처리 영역까지 확장된 Vision Transformer의 등장까지 이어졌습니다. 본 학습서는 이러한 기술적 진화 과정을 체계적으로 설명하여 Vision Transformer의 핵심 원리를 완전히 이해할 수 있도록 구성되었습니다.

RNN → 워드 임베딩 → 인코더-디코더 → 어텐션 메커니즘 → 트랜스포머 →
Vision Transformer

1. RNN (순환 신경망)

1.1 RNN의 기본 구조

순환 신경망(Recurrent Neural Network)은 시퀀스 데이터를 처리하기 위해 설계된 신경망으로, 이전 시점의 정보를 현재 시점으로 전달하는 메모리 기능을 가지고 있습니다.

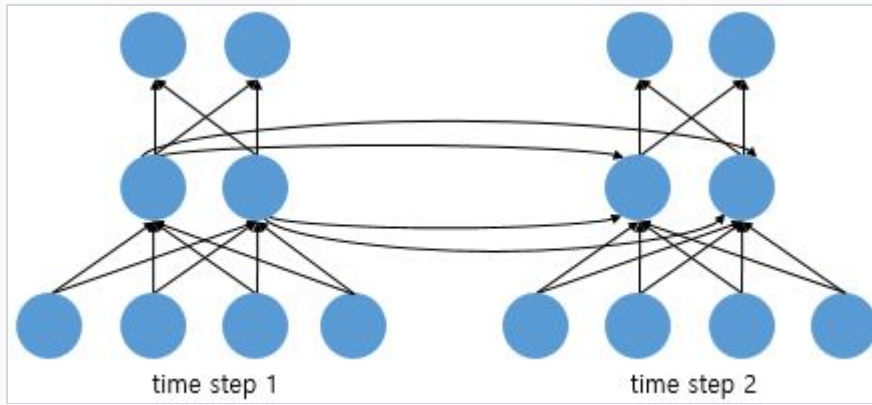
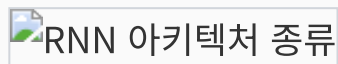


그림 1: RNN의 시간 단계별 처리 과정 (time step 1, time step 2)

RNN의 핵심 특징:

- 시간 단계별로 순차적 처리
- 이전 상태(hidden state) 정보를 다음 단계로 전달
- 가변 길이 시퀀스 처리 가능

1.2 RNN의 다양한 구조



RNN 아키텍처 종류

그림 2: RNN의 다양한 아키텍처 (일대다, 다대일, 다대다)

RNN은 입력과 출력의 관계에 따라 다음과 같이 분류됩니다:

- 일대다(One-to-Many) : 하나의 입력으로 여러 출력 생성 (예: 이미지 캡셔닝)
- 다대일(Many-to-One) : 여러 입력을 하나의 출력으로 변환 (예: 스팸메일 판별, 감정 분석)
- 다대다(Many-to-Many) : 여러 입력을 여러 출력으로 변환 (예: 기계번역, 개체명 인식)

스팸메일 판별 예시 (다대일 구조):

입력: "무료", "혜택", "클릭", "지금" → RNN 처리 → 출력: 스팸 확률 0.95

2. 워드 임베딩 (Word Embedding)

2.1 텍스트 데이터의 수치화

자연어 처리에서 컴퓨터가 텍스트를 이해하려면 단어를 숫자로 변환해야 합니다. **코퍼스(Corpus)** 는 자연어 처리를 위해 수집된 텍스트 데이터의 집합을 의미합니다.

2.2 원-핫 벡터 (One-Hot Vector)

원-핫 벡터는 단어를 벡터로 표현하는 가장 기본적인 방법으로, 머신러닝의 라벨 바이너라이제이션(Label Binarization)과 유사한 개념입니다.

원-핫 벡터 예시:

사전: ["사과", "바나나", "오렌지"]

"사과" → [1, 0, 0]

"바나나" → [0, 1, 0]

"오렌지" → [0, 0, 1]

2.3 원-핫 벡터의 한계

- **차원의 저주:** 단어가 늘어날수록 벡터의 크기가 기하급수적으로 증가
- **희소성 문제:** 대부분의 값이 0인 희소 표현(Sparse Representation)
- **의미적 관계 부족:** 단어 간의 유사성이나 관계를 표현할 수 없음

2.4 밀집 표현 (Dense Representation)

이러한 문제를 해결하기 위해 **밀집 표현** 이 등장했습니다. 정수나 바이너리가 아닌 실수 벡터를 사용하여 단어를 표현하며, 일반적으로 128차원 정도의 벡터를 사용합니다.

워드 임베딩:

단어를 밀집 벡터로 변환하는 과정

임베딩 벡터:

워드 임베딩 과정을 통해 생성된 실수 벡터

2.5 토큰과 임베딩의 차이

- 토큰(Token): 텍스트를 최소 의미 단위로 나눈 것 (예: 단어, 형태소)
- 임베딩(Embedding): 토큰을 수치 벡터로 변환한 결과

3. 인코더-디코더 구조 (Encoder-Decoder Architecture)

3.1 시퀀스-투-시퀀스 모델

인코더-디코더 구조는 입력 시퀀스를 다른 시퀀스로 변환하는 모델로, 기계번역과 같은 작업에 사용됩니다.



그림 3: 기계번역에서의 시퀀스-투-시퀀스 변환

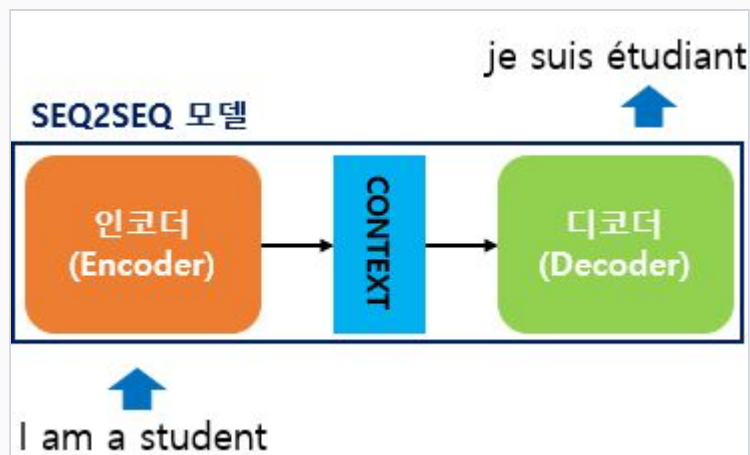


그림 4: 인코더-컨텍스트-디코더 구조

3.2 컨텍스트 벡터 (Context Vector)

컨텍스트 벡터는 인코더가 입력 시퀀스의 모든 정보를 압축하여 생성한 고정 크기의 벡터입니다.

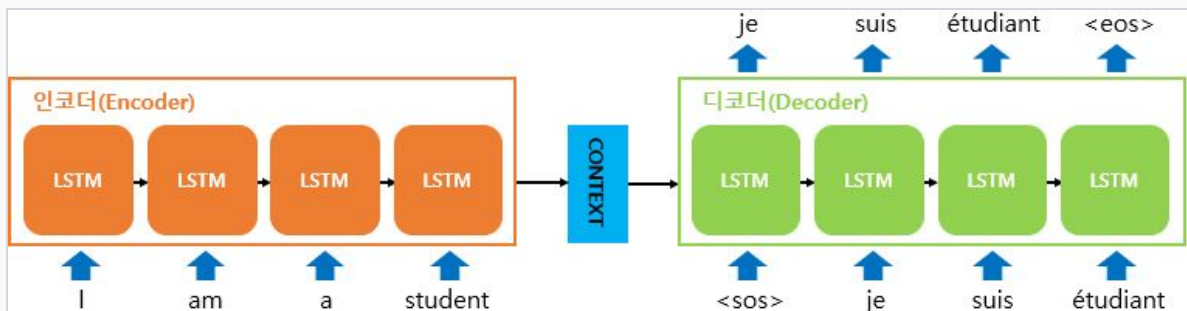


그림 5: LSTM 기반 인코더-디코더의 상세 동작 과정

3.3 인코더-디코더 동작 원리

인코더의 역할:

1. 입력 문장을 단어별로 순차 처리
2. 각 단계에서 히든 스테이트 업데이트
3. 최종 히든 스테이트를 컨텍스트 벡터로 출력

디코더의 역할:

1. 컨텍스트 벡터를 초기 상태로 받음

2. <sos> 토큰부터 시작하여 단어 생성
3. 이전 출력을 다음 입력으로 사용
4. <eos> 토큰까지 반복 생성

4. 어텐션 메커니즘 (Attention Mechanism)

4.1 컨텍스트 벡터의 한계

기존 인코더-디코더 구조는 모든 입력 정보를 고정 크기의 컨텍스트 벡터에 압축해야 하는 병목 현상(Bottleneck)이 있었습니다. 특히 긴 문장에서는 중요한 정보가 손실될 수 있습니다.

4.2 어텐션의 핵심 아이디어

어텐션 메커니즘은 디코더가 출력을 생성할 때마다 입력 시퀀스의 모든 위치를 다시 참조할 수 있게 합니다. 이때 어텐션 가중치(Attention Weights)를 통해 현재 출력과 관련성이 높은 입력 부분에 더 집중할 수 있습니다.

어텐션 계산 과정:

1. 디코더의 현재 상태와 인코더의 모든 상태 비교
2. 유사도 계산 (점수 함수 사용)
3. 소프트맥스를 통한 어텐션 가중치 계산
4. 가중합으로 컨텍스트 벡터 생성
5. 컨텍스트 벡터와 디코더 상태 결합하여 출력 생성

4.3 어텐션의 장점

- 정보 손실 방지: 모든 입력 정보에 직접 접근 가능
- 긴 시퀀스 처리: 문장 길이에 관계없이 성능 유지

- **해석가능성**: 어텐션 가중치를 통한 모델 동작 이해
- **정렬 학습**: 입력과 출력 간의 정렬 관계 자동 학습

5. 트랜스포머 (Transformer)

5.1 RNN 없는 어텐션 기반 모델

트랜스포머는 "Attention Is All You Need" 논문에서 제안된 구조로, RNN이나 LSTM을 전혀 사용하지 않고 오직 어텐션 메커니즘만으로 시퀀스를 처리합니다.

5.2 셀프 어텐션 (Self-Attention)

셀프 어텐션은 같은 시퀀스 내의 서로 다른 위치에 있는 토큰들 간의 관계를 모델링합니다. 각 토큰이 자기 자신을 포함한 모든 토큰과의 관련성을 계산합니다.

셀프 어텐션 계산:

$$Q \text{ (Query)} = X \times W_Q$$

$$K \text{ (Key)} = X \times W_K$$

$$V \text{ (Value)} = X \times W_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

5.3 멀티-헤드 어텐션 (Multi-Head Attention)

여러 개의 어텐션 헤드를 병렬로 사용하여 서로 다른 측면의 정보를 동시에 학습합니다. 각 헤드는 다른 가중치 행렬을 사용하여 독립적으로 어텐션을 계산합니다.

5.4 위치 인코딩 (Positional Encoding)

트랜스포머는 RNN과 달리 순차적 처리를 하지 않으므로, 토큰의 위치 정보를 별도로 제공해야 합니다. 위치 인코딩은 사인(sin)과 코사인(cos) 함수를 사용하여 각 위치에 고유한 벡터를 할당합니다.

5.5 트랜스포머의 인코더-디코더 구조

인코더: 입력 임베딩 + 위치 인코딩 → 멀티-헤드 어텐션 → 피드포워드 → 출력
디코더: 출력 임베딩 + 위치 인코딩 → 마스크드 어텐션 → 인코더-디코더 어텐션
→ 피드포워드 → 최종 출력

5.6 트랜스포머의 장점

- 병렬 처리: RNN과 달리 모든 토큰을 동시에 처리 가능
- 장거리 의존성: 거리에 관계없이 직접적인 연결
- 확장성: 큰 모델과 데이터셋에서 우수한 성능
- 전이 학습: 사전 훈련된 모델의 활용 가능

6. Vision Transformer (ViT)

6.1 이미지에 트랜스포머 적용

Vision Transformer는 "An Image is Worth 16x16 Words" 논문에서 제안된 방법으로, 이미지를 패치로 나누어 시퀀스처럼 처리 하여 트랜스포머를 이미지 분류에 적용합니다.

6.2 패치 임베딩 (Patch Embedding)

ViT의 핵심 아이디어는 이미지를 작은 패치들로 나누는 것입니다. 예를 들어, 224×224 이미지를 16×16 크기의 패치로 나누면 총 196개의 패치를 얻을 수 있습니다.

패치 분할 과정:

1. 입력 이미지 ($H \times W \times C$) 준비
2. $P \times P$ 크기의 패치로 분할

3. 각 패치를 1차원 벡터로 변환
4. 선형 변환을 통해 임베딩 차원으로 투영

6.3 이미지 패치를 토큰으로 처리

분할된 이미지 패치들은 NLP에서의 단어 토큰과 동일하게 취급됩니다. 각 패치가 하나의 토큰 역할을 하며, 시퀀스의 한 요소로 처리됩니다.

6.4 ViT 아키텍처

이미지 입력 → 패치 추출 → 선형 임베딩 → 위치 임베딩 추가 → [CLS] 토큰 추가 → 트랜스포머 인코더 → 분류 헤드 → 최종 예측

ViT의 주요 구성 요소:

- 패치 임베딩: 이미지 패치를 벡터로 변환
- 위치 임베딩: 각 패치의 공간적 위치 정보
- [CLS] 토큰: 분류를 위한 특별한 토큰
- 트랜스포머 인코더: 멀티-헤드 어텐션과 MLP
- 분류 헤드: 최종 클래스 예측을 위한 선형 층

6.5 ViT의 특징과 장점

- **글로벌 정보 처리:** 첫 번째 레이어부터 전체 이미지 정보 활용
- **유연한 해상도:** 다양한 입력 해상도에 적응 가능
- **해석가능성:** 어텐션 맵을 통한 모델 동작 이해
- **확장성:** 모델 크기와 데이터에 따른 성능 향상

6.6 훈련 요구사항과 성능

ViT는 CNN에 비해 **귀납적 편향(Inductive Bias)** 이 적어 대용량 데이터셋에서 사전 훈련이 필요합니다. 하지만 충분한 데이터와 컴퓨팅 자원이 있을 때 CNN을 능가하는

성능을 보입니다.

ViT 성능 특성:

- 소규모 데이터: $CNN < ViT$
- 대규모 데이터: $CNN < ViT$
- ImageNet-21k 사전훈련 시 ImageNet-1k에서 SOTA 달성
- 계산 효율성: 동일 성능 대비 CNN보다 적은 계산량

6.7 ViT의 변형과 발전

ViT 이후 다양한 변형 모델들이 등장했습니다:

- **DeiT**: 지식 증류를 통한 효율적 훈련
- **Swin Transformer**: 계층적 구조와 슬라이딩 윈도우
- **PVT**: 피라미드 구조의 비전 트랜스포머
- **CaiT**: 클래스 어텐션을 분리한 구조

결론

Vision Transformer는 자연어 처리 분야에서 시작된 트랜스포머 아키텍처를 컴퓨터 비전 영역으로 성공적으로 확장한 혁신적인 모델입니다. RNN의 순차 처리 방식에서 시작하여 어텐션 메커니즘을 거쳐 트랜스포머로 발전한 과정을 이해함으로써, 현재의 ViT가 어떻게 이미지를 효과적으로 처리할 수 있는지 명확히 파악할 수 있습니다.

ViT의 성공은 단순히 아키텍처의 우수성뿐만 아니라, 충분한 데이터와 컴퓨팅 자원이 뒷받침될 때 도메인에 특화된 귀납적 편향보다 범용적인 학습 능력이 더 중요할 수 있음을 시사합니다. 이는 앞으로의 딥러닝 연구 방향에도 중요한 통찰을 제공합니다.